

Hands-On Session

Q-Learning on the Gridworld

Large Scale Optimization (LSO) Summer School 2026 · IIT Delhi

June 1–6, 2026

Objective

By the end of this session you will have **trained a Q-learning agent** to navigate a Gridworld from scratch. Along the way you will observe — empirically — how each hyperparameter affects learning speed, policy quality, and convergence behaviour. No prior coding of RL algorithms is required: the infrastructure is provided. **Your job is to experiment, observe, and reflect.**

1. Files Provided

You have been given two Python files. Keep them in the **same folder**.

File	Purpose
<code>gridworld.py</code>	Defines the Gridworld <i>environment</i> . You will not need to modify this file, but you are encouraged to read it.
<code>qlearning.py</code>	Runs the Q-learning algorithm. This is the file you will work with. All hyperparameters are collected at the top of the file with comments.

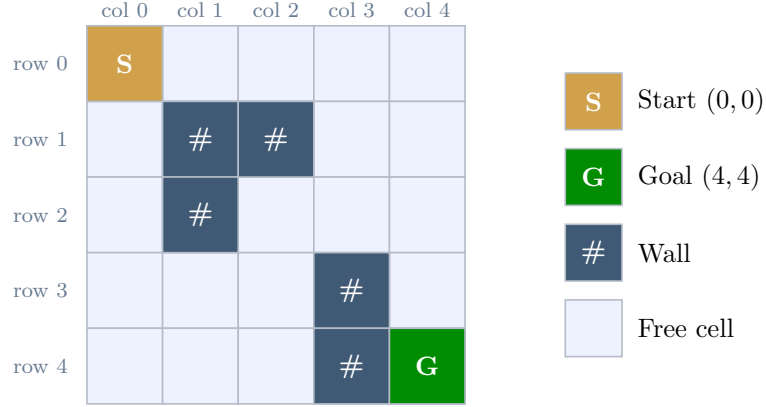
Requirements: Python 3.8+, numpy, matplotlib. Install with:

```
pip install numpy matplotlib
```

2. The Gridworld Environment

2.1. Layout

The default environment is a 5×5 grid. The agent starts at cell **S** (0,0) (top-left) and must reach the goal **G** (4,4) (bottom-right). Several cells are **walls** that cannot be entered.



2.2. States, Actions, and Transitions

- **State** s : the agent's position (r, c) , encoded as a flat index $s = r \times 5 + c \in \{0, 1, \dots, 24\}$. There are 25 states in total.
- **Actions**: four movement directions — 0 = $\uparrow$ (UP), 1 = $\downarrow$ (DOWN), 2 = $\leftarrow$ (LEFT), 3 = $\rightarrow$ (RIGHT).
- **Transitions**: the agent moves in the chosen direction. If the target cell is a wall or outside the grid, the agent *stays put*. With `slip_prob` > 0, the intended action is replaced by a uniformly random one with that probability (stochastic variant).

2.3. Reward Structure

Event	Reward
Reaching the goal cell (4, 4)	+10
Any other step (including hitting a wall)	-0.1

The step penalty discourages the agent from wandering and incentivises finding a **short** path to the goal. The episode ends as soon as the goal is reached, or after `MAX_STEPS` steps.

3. The Q-Learning Algorithm (Recap)

Q-learning maintains a table $Q(s, a) \in \mathbb{R}$ for every state-action pair. After each transition (s_t, a_t, r_t, s_{t+1}) it performs:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \underbrace{\alpha}_{\text{learning rate}} \underbrace{\left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]}_{\text{TD error } \delta_t}$$

TD target

Actions are chosen via an ϵ -greedy policy:

$$a_t = \begin{cases} \text{random action} & \text{with probability } \epsilon \quad (\text{explore}) \\ \arg \max_a Q(s_t, a) & \text{otherwise} \quad (\text{exploit}) \end{cases}$$

The exploration rate ε is decayed over episodes so that the agent explores broadly early on and gradually shifts to exploiting its learned knowledge.

4. Running the Code

Step 1. Place `gridworld.py` and `qlearning.py` in the same folder.

Step 2. Open `qlearning.py` in any text editor or IDE.

Step 3. Run it from the terminal:

```
python qlearning.py
```

Step 4. Two windows will appear:

- A **5-panel figure** showing the training curves, exploration schedule, greedy policy performance, steps to goal, and a heatmap of learned values.
- The **Gridworld grid** with arrows showing the final greedy policy.

Step 5. A file `qlearning_results.png` is also saved in the same folder.

5. Hyperparameters to Tune

All parameters are in the `HYPERPARAMETERS` block at the top of `qlearning.py`. The table below summarises what each one does.

Parameter	Default	What it controls
ALPHA	0.3	Learning rate $\alpha \in (0, 1]$. How strongly each update overwrites the current estimate.
GAMMA	0.95	Discount factor $\gamma \in [0, 1)$. How much future rewards are valued relative to immediate ones.
N_EPISODES	600	Total number of training episodes.
MAX_STEPS	100	Maximum steps per episode before it is forcibly terminated.
EPSILON_START	1.0	Initial exploration rate (1 = fully random).
EPSILON_END	0.05	Final (minimum) exploration rate.
EPSILON_DECAY	"exp"	Decay schedule: "exp" (exponential) or "linear".
SEED	42	Random seed for reproducibility.

6. Guided Experiments

Work through the experiments below in order. For each one, **change only the stated parameter(s)**, run the code, and record your observations in the space provided. Use the guiding questions to direct your thinking.

Experiment A — Effect of the Learning Rate α

Change: ALPHA. Keep everything else at default.

Run 1: ALPHA = 0.01

Run 2: ALPHA = 0.3 (default)

Run 3: ALPHA = 0.9

Observe: How smooth is the training reward curve? Does the policy converge to the same quality in all three runs? Does a large α cause instability?

(Write your observations here.)

Experiment B — Effect of the Discount Factor γ

Change: GAMMA. Keep everything else at default.

Run 1: GAMMA = 0.5

Run 2: GAMMA = 0.9

Run 3: GAMMA = 0.99

Observe: Look at the “steps to goal” plot and the policy arrows on the grid. Does a higher γ lead to shorter paths? Why might a very low γ cause the agent to give up on reaching the goal?

(Write your observations here.)

Experiment C — Effect of Number of Episodes

Change: N_EPISODES. Keep everything else at default.

Run 1: N_EPISODES = 50

Run 2: N_EPISODES = 200

Run 3: N_EPISODES = 600 (default)

Observe: Does the agent find the goal with only 50 episodes? At approximately which episode does the greedy policy stabilise? Does training beyond that point help further?

(Write your observations here.)

Experiment D — Effect of Episode Length

Change: MAX_STEPS. Keep everything else at default.

Run 1: MAX_STEPS = 10

Run 2: MAX_STEPS = 30

Run 3: MAX_STEPS = 100 (default)

Observe: With very short episodes, can the agent ever reach the goal early in training? How does this interact with the exploration rate? What is the minimum episode length that still allows convergence?

(Write your observations here.)

Experiment E — Exploration vs. Exploitation

Change: EPSILON_START, EPSILON_END, EPSILON_DECAY. Keep everything else at default.

Run 1: EPSILON_START = 0.05, EPSILON_END = 0.05 (almost no exploration)

Run 2: EPSILON_START = 1.0, EPSILON_END = 0.05, EPSILON_DECAY = "exp"

Run 3: EPSILON_START = 1.0, EPSILON_END = 0.5 (too much exploration)

Run 4: EPSILON_START = 1.0, EPSILON_END = 0.05 EPSILON_DECAY = "linear"

Observe: Does Run 1 converge? Compare the greedy policy reward in Runs 2 and 3. What does the exploration schedule plot look like for "exp" vs "linear" decay?

(Write your observations here.)

Experiment F — Stochastic Environment

Change: slip_prob in the GridWorld() constructor (bottom of the ENVIRONMENT section in qlearning.py). Keep all other parameters at default.

Run 1: slip_prob = 0.0 (deterministic, default)

Run 2: slip_prob = 0.1

Run 3: slip_prob = 0.3

Observe: Does the algorithm still converge under noise? Does the agent need more episodes? Does the final policy look different — does it avoid states near walls more cautiously?

(Write your observations here.)

7. Discussion Questions

After completing the experiments, discuss the following questions with your neighbours, and the members of the teaching staff.

- Q1.** The Q-learning update is often described as *stochastic fixed-point iteration*. After completing Experiment A, does this description make more sense? What role does α play in the analogy?
- Q2.** In Experiment B, why does a low discount factor γ make the agent “short-sighted”? Can you construct an example where being short-sighted leads to a strictly worse policy in this grid?
- Q3.** The Bellman optimality equation uses a max operator. Q-learning replaces this expectation with a single sample. Does this introduce bias? Does it matter in practice?
- Q4.** In Experiment F, the environment becomes stochastic. Q-learning still converges — why? What theoretical condition on the step sizes $\{\alpha_t\}$ guarantees convergence under noise?
- Q5.** How would you scale this approach to a 100×100 grid, or to a continuous state space (e.g. a robot arm)? What breaks, and how does Deep Q-Learning (DQN) address it?

8. Quick Reference

Action encoding

Code	Direction
0	↑ UP
1	↓ DOWN
2	← LEFT
3	→ RIGHT

State encoding

State index $s = r \times 5 + c$ where (r, c) is (row, column).

(r, c)	s
(0, 0)	0 (Start)
(0, 4)	4
(4, 0)	20
(4, 4)	24 (Goal)

Gridworld API (from gridworld.py)

```
1 from gridworld import GridWorld
2
3 env = GridWorld(slip_prob=0.0)      # create environment
4 s = env.reset()                     # reset; returns flat state index
5 s_next, reward, done = env.step(action) # take action (0-3)
6 env.render(policy=None)             # visualise grid (+ policy arrows)
```

Reminder: Only modify `qlearning.py` (the `HYPERPARAMETERS` and `ENVIRONMENT` blocks). Do not change `gridworld.py` unless you are exploring bonus extensions.

Bonus (if you finish early)

- B1. Change the grid.** In `gridworld.py`, modify `DEFAULT_WALLS` to add or remove walls and re-run. Does the learned policy adapt correctly?
 - B2. Change the reward.** Set `step_penalty = -1.0`. Does the agent become more aggressive about finding the goal quickly?
 - B3. Larger grid.** Try `n_rows=7`, `n_cols=7`, `goal=(6,6)`. How many more episodes does convergence require?
 - B4. Visualise Q-values.** After training, print `Q.reshape(5,5,4)` and inspect the values for a few states near the walls. Do the Q-values make intuitive sense?
-